Lista de Revisão – STRUCTS

1. Somar Pontos Cardeais

Defina uma struct chamada `Ponto` contendo:

- `int x`
- `int y`

Implemente uma função chamada `somar_pontos`. A função deve receber como parâmetros:

- duas structs do tipo `Ponto`

A função deve retornar:

- uma struct `Ponto` com a soma das coordenadas:
  - x = x1 + x2
  - y = y1 + y2

Protótipo da função:

```
Ponto somar_pontos(Ponto a, Ponto b);
```

Implemente uma função `main()` que:

1. leia dois pontos;
2. chame a função `somar_pontos`;
3. imprima o ponto resultante.

2. Cadastro de Aluno com Média

Defina uma struct chamada `Aluno` contendo:

- `char nome[50]`
- `float nota1`
- `float nota2`
- `float nota3`

Implemente uma função chamada `calcular_media_aluno`. A função deve receber como parâmetro:

- uma struct do tipo `Aluno`

A função deve retornar a média aritmética das três notas do aluno.

Implemente uma função chamada `buscar_melhor_aluno`. A função deve receber como parâmetros:

- um vetor de structs do tipo `Aluno`
- um inteiro n

A função deve retornar o **índice** do aluno com maior média. Em caso de empate, retorne o primeiro encontrado.

Protótipos:

```
float calcular_media_aluno(Aluno a);

int buscar_melhor_aluno(Aluno v[], int n);
```

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de `Aluno` com n posições;
3. leia o nome e as três notas de cada aluno;
4. chame `buscar_melhor_aluno`;
5. imprima o nome e a média do aluno encontrado.

3. Produto Mais Caro

Defina uma struct chamada `Produto` contendo:

- `char nome[50]`
- `float preco`
- `int quantidade`

Implemente uma função chamada `buscar_produto_mais_caro`.

A função deve receber como parâmetros:

- um vetor de structs do tipo `Produto`;
- um inteiro n, representando a quantidade de produtos no vetor.

A função deve retornar:

- a posição do produto com maior preço no vetor.

Considere que:

- se houver empate, deve ser retornada a primeira posição encontrada.

Protótipo da função:

int buscar_produto_mais_caro(Produto v[], int n);

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de `Produto` com n posições;
3. leia os dados de todos os produtos;
4. chame a função `buscar_produto_mais_caro`;
5. imprima o nome, o preço e a quantidade do produto encontrado.

4. Funcionário com Maior Salário Total

Defina uma struct chamada `Funcionario` contendo:

- char nome[50]
- float salario_base
- float bonus

Implemente uma função chamada `buscar_maior_total`. A função deve receber como parâmetros:

- um vetor de structs do tipo Funcionario;
- um inteiro n

A função deve retornar:

- a posição do funcionário com maior salário total (salario_base + bonus)

Considere que:

- em caso de empate, deve ser retornada a primeira posição encontrada.

Protótipo da função:

```
int buscar_maior_total(Funcionario v[], int n);
```

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de `Funcionario` com n posições;
3. leia os dados de todos os funcionários;
4. chame a função `buscar_maior_total`;
5. imprima o nome e o salário total do funcionário encontrado.

5. Média da Turma

Defina uma struct chamada `Aluno` contendo:

- char nome[50]
- float notas[4]

Implemente uma função chamada `media_turma`. A função deve receber como parâmetros:

- um vetor de structs do tipo `Aluno`;
- um inteiro n

A função deve retornar:

- a média geral da turma

Considere que:

- a média de cada aluno é a média das 4 notas.

Protótipo da função:

```
float media_turma(Aluno v[], int n);
```

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de Aluno com n posições;
3. leia o nome e as 4 notas de cada aluno;
4. chame a função media_turma;
5. imprima a média da turma.

6. Sistema de Retângulos

Defina as structs abaixo:

```
typedef struct {
  float x;
  float y;
```

```
} Ponto;

typedef struct {
  Ponto p1;
  Ponto p2;
} Retangulo;
```

Implemente as funções:

```
float area_retangulo(Retangulo r);
int contar_retangulos_maiores(Retangulo v[], int n, float valor);
```

A função `area_retangulo` deve:

- receber uma struct do tipo `Retangulo`;
- retornar a área do retângulo.

Considere que:

- a base é dada pelo **valor absoluto** da diferença entre as coordenadas `x`;
- a altura é dada pelo **valor absoluto** da diferença entre as coordenadas `y`.

A função `contar_retangulos_maiores` deve:

- receber um vetor de `Retangulo`, um inteiro n e um valor real;
- retornar a quantidade de retângulos cuja área é maior que o valor informado.

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de `Retangulo` com n posições;
3. leia os dados de cada retângulo;
4. leia um valor real;
5. chame a função `contar_retangulos_maiores`;
6. imprima a quantidade retornada.

7. Maior Distância entre Pontos

Defina uma struct chamada `Ponto` contendo:

- float x
- float y

Implemente uma função chamada `maior_distancia`. A função deve receber como parâmetros:

- um vetor de structs do tipo `Ponto`;
- um inteiro n

A função deve retornar:

- o maior valor de distância entre dois pontos do vetor

Considere que:

- a distância entre dois pontos é dada por:
  - $\sqrt{((x1 - x2)^2 + (y1 - y2)^2)}$

Protótipo da função:

```
float maior_distancia(Ponto v[], int n);
```

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de Ponto com n posições;
3. leia as coordenadas de cada ponto;
4. chame a função maior_distancia;
5. imprima o valor retornado.

8. Cadastro de Pessoas com Data

Defina as structs abaixo:

```
typedef struct {
  int dia;
  int mes;
  int ano;
} Data;

typedef struct {
  char nome[50];
  Data nascimento;
} Pessoa;
```

Implemente uma função chamada `ordenar_por_idade`.

A função deve receber como parâmetros:

- um vetor de structs do tipo `Pessoa`;
- um inteiro n.

A função deve:

- ordenar o vetor **na própria memória**, da pessoa mais velha para a mais nova.

Considere que:

- a comparação deve ser feita primeiro pelo ano;
- em caso de empate, pelo mês;
- persistindo o empate, pelo dia.

Restrição:

- não utilizar funções prontas de ordenação.

Protótipo da função:

void ordenar_por_idade(Pessoa v[], int n);

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de `Pessoa` com n posições;
3. leia o nome e a data de nascimento de cada pessoa;
4. chame a função `ordenar_por_idade`;
5. imprima o vetor já ordenado.

9. Sistema de Pedidos

Defina as structs abaixo:

```
typedef struct {

    char nome[50];

    float preco;

} Produto;



typedef struct {

     Produto itens[5];

     int quantidade;

} Pedido;
```

Implemente as funções:

- float valor_total_pedido(Pedido p);
- int pedido_mais_caro(Pedido v[], int n);

A função `valor_total_pedido` deve:

- receber uma struct do tipo Pedido;
- retornar a soma dos preços dos produtos do pedido.

A função `pedido_mais_caro` deve:

- receber um vetor de Pedido e um inteiro n;
- retornar a posição do pedido com maior valor total.

Considere que:

- em caso de empate, deve ser retornada a primeira posição encontrada.

Implemente uma função `main()` que:

1. leia um inteiro n;
2. declare um vetor de Pedido com n posições;
3. leia os dados de cada pedido (quantidade e produtos);
4. chame a função `pedido_mais_caro`;
5. imprima o valor total do pedido encontrado.

10. Sistema de Biblioteca com Struct Aninhada

Defina as structs abaixo:

```
typedef struct {
  char titulo[50];
  int ano;
} Livro;

typedef struct {
  char nome[50];
  int quantidade_livros;
  Livro livros[10];
} Autor;
```

Implemente uma função chamada `autor_com_livro_mais_recente`.

A função deve receber como parâmetros:

- um vetor de structs do tipo `Autor`;
- um inteiro `n`.

A função deve:

- percorrer todos os autores do vetor;
- percorrer todos os livros de cada autor até o limite indicado por quantidade_livros;
- identificar o livro com o maior ano de publicação;
- retornar a struct completa do autor dono desse livro.

Considere que:

- todo autor possui pelo menos 1 livro;
- não é permitido usar funções prontas;
- não é permitido criar structs auxiliares.

Protótipo da função:

Autor autor_com_livro_mais_recente(Autor v[], int n);

Implemente uma função `main()` que:

1. leia um inteiro `n`;
2. declare um vetor de `Autor` com n posições;
3. leia o nome de cada autor;
4. leia a quantidade de livros de cada autor;
5. leia o título e o ano de cada livro;
6. chame a função `autor_com_livro_mais_recente`;
7. imprima o nome do autor retornado.